Project 2 A* 栅格路径规划设计与算法分析说明书

1 任务与交付内容

本项目使用 Python 在二维栅格地图上实现 A* 最短路径搜索，支持多种启发式函数和拓展策略。采用单文件代码，通过函数划分地图处理、搜索、输入输出和绘图。

提交材料为以下三个文件；设计说明书.md 保留为说明书的可编辑文本版本：

文件	用途
astar.py	完整源代码，包含中文注释、示例地图、A* 算法及结果绘图
设计说明书（更新版）.docx	设计思路、算法原理、实现细节、实验结果和优缺点
运行结果.png	实际运行生成的路径图与策略比较表

2 地图与路径定义

地图表示为二维数组 `grid[row][col]`，整数 0 表示可通行格，整数 1 表示障碍物。坐标采用 (行, 列)，从 0 开始，行号向下增大、列号向右增大。

代码接口接受由普通 Python 列表或元组构成的地图与坐标；格子值和坐标分量必须为内置 int。bool、NumPy 整数以及 NumPy 数组不在支持范围内。JSON 中的整数和数组会分别转换为 Python 整数和列表，可直接使用。

输入包括地图、起点和目标点。输出为包含起点和目标点的坐标序列，同时给出路径步数、累计移动代价、扩展节点数和搜索耗时。最优路径可能不唯一。

支持两种邻域拓展方式：

- **四邻域**：上下左右移动，每步代价为 1。
- **八邻域**：增加四个对角方向，正交移动代价为 1，对角移动代价为 $\sqrt{2}$。对角移动时，两个相邻正交格必须同时可通行，防止从障碍物夹角穿过。

“最短”指累计移动代价最小。八邻域中步数最少不必然等于代价最小。

3 A* 算法原理

A* 用以下估价函数选择下一个待拓展节点：

$$f(n) = g(n) + h(n)$$

其中，$g(n)$ 是目前找到的起点至节点 n 的最小路径代价，$h(n)$ 是节点 n 到目标的估计代价，$f(n)$ 是经过节点 n 到达目标的估计总代价。启发式满足可采纳性，即不高估真实剩余代价；本程序支持的组合还满足一致性，即每条合法边都满足 $h(u) \leq cost(u,v) + h(v)$。

搜索流程：

1. 起点的 g 设为 0，计算 h，将起点加入候选堆。
2. 取出 f 最小的条目；如果条目保存的 g 已过期，则跳过。
3. 若当前节点是目标，沿父节点回溯并反转，得到完整路径。
4. 否则枚举合法邻居，计算候选代价 $new_g = g(current) + step_cost$。
5. 若 new_g 更小，则更新该邻居的 g、父节点和堆条目。
6. 重复搜索；堆为空仍未到达目标，则判定无路可达。

算法在目标有效出堆时终止，而不是在首次发现目标时终止。程序允许改进后的节点重新入堆，没有使用禁止重新访问的永久封闭集合。对题目中的静态栅格、非负移动代价及本程序支持的启发式组合，该设计保证最短路径搜索的正确性；计算使用双精度浮点数，对角代价核对允许合理的浮点误差。

4 多种启发式函数与拓展策略

设 $dx = |row-row_goal|$，$dy = |col-col_goal|$。

名称	计算公式	可用邻域
零启发式 zero	0，此时 A* 等价于 Dijkstra	四、八
曼哈顿 manhattan	dx + dy	四
欧氏 euclidean	$\sqrt{(dx^2 + dy^2)}$	四、八
切比雪夫 chebyshev	max(dx, dy)	四、八
Octile octile	$\max(dx,dy) + (\sqrt{2}-1) \times \min(dx,dy)$	八

四邻域默认使用曼哈顿距离，八邻域默认使用 Octile 距离。它们分别等于对应模型在无障碍地图上的最短代价。障碍物只会限制路线，不会使真实最短代价变得更小，因此这些估计仍是下界。

八邻域禁止选择曼哈顿距离。例如目标相距一个对角格时，曼哈顿估计为 2，但合法对角移动仅需 √2，产生高估。程序会拒绝不支持的组合。Octile 对四邻域也是可用下界，但本程序仅将其开放给八邻域，减少冗余配置。

除四 / 八邻域外，代码还支持两种同优先级拓展策略：

- `low_h`：f 相同，优先选择 h 较小的节点；仍相同则按入队顺序。
- `fifo`：f 相同，直接按入队顺序。

两者都以 f 为首要优先级，不改变最短路径目标。它们可能影响路径形状和扩展节点数，不能保证某一种在所有地图上都更快。代码按原始浮点 f 排序，不将“非常接近”的值人为视为相同。

这些配置属于同一 A* 算法的不同启发式和拓展策略。四邻域、八邻域各有四种合法启发式，每种再搭配两种同优先级策略，共 16 组合法配置。生成结果图时会运行全部组合，终端默认只展示两组预设配置，并不先比较所有配置再筛选所谓“最优方案”。

5 单文件中的功能划分

函数 / 类型	职责
SearchResult	保存路径、代价、扩展顺序和耗时
validate	校验矩形地图、0/1 格子、整数坐标、越界和障碍物起终点
estimate	计算选定的启发式值
neighbors	生成合法邻居及移动代价，落实对角防穿角规则
astar	堆搜索、代价更新、过期条目处理和路径回溯
demo_map	生成默认 15×20 示例地图及起终点
draw_results	将实际搜索结果和策略比较绘制为一张图片
main	命令行参数、可选 JSON 输入、终端结果输出和错误提示

核心数据结构为 `heapq` 最小堆、`g_score` 代价字典、`parent` 父节点字典和 `expanded` 扩展记录。堆的排序字段为 f、次级优先级和唯一入队序号，不依赖坐标大小决定优先级。

输入地图不会被搜索修改。绘图复制地图后再标记已扩展格。搜索耗时从初始化候选堆前开始，到返回搜索结果为止，不包含输入校验、读文件和绘图。

无路可达时返回空路径和 None 代价，终端明确显示状态；起点等于目标时返回单点路径，代价为 0。合法但不可达属于正常搜索结果，程序正常退出。

命令行先检查启发式与邻域组合，再检查需要绘图时的输出路径；读取地图后，先执行输入校验，才显示地图尺寸、转换坐标和开始搜索。空地图、非法坐标等输入会给出可读错误并以非零状态退出，避免在校验前读取数据引发未处理异常。astar 函数也保留独立校验，便于从其他 Python 代码直接调用。

6 安装与运行

需要 Python 3.10 或更高版本。核心搜索只使用标准库；生成图片需要 Matplotlib。

在三个文件所在目录打开终端，执行：

```
python -m pip install matplotlib
python astar.py
```

默认运行在终端分别输出四邻域（曼哈顿、low_h）和八邻域（Octile、low_h）的坐标路径、步数、总代价、扩展节点数及搜索耗时。显式选择邻域或启发式时，终端仅输出选定模型；--tie 同时控制默认两组搜索的同优先级策略。

单独选择启发式时按四邻域处理。因此，使用 --heuristic octile 必须同时指定 --movement 8，否则程序明确提示这一要求；八邻域不接受 manhattan。可通过 python astar.py --help 查看参数规则。

默认将图片写入脚本所在目录的 运行结果.png，每次成功绘图都会覆盖同名文件。需要保留不同运行结果时，使用 --output 指定不同文件名；相对路径以终端当前工作目录为准。程序在搜索前检查输出父目录存在且输出路径不是目录。使用 --no-plot 时不检查图片路径，也不更新已有图片。

图片为 2340×1800 像素。中文字体优先使用 Windows 的 Microsoft YaHei，其次 SimHei；其他系统若没有中文字体，需安装字体或调整代码中的字体列表。

如果 VS Code 使用的 Python 缺少绘图依赖，搜索结果仍正常输出，程序跳过本次图片生成，并显示当前 Python 路径及对应的 PowerShell 安装命令。请执行显示的命令再运行，避免将 Matplotlib 安装到另一个 Python 环境。跳过绘图时不会更新已有的图片。

选择八邻域，或比较入队顺序策略：

```
python astar.py --movement 8 --heuristic octile
python astar.py --movement 4 --heuristic manhattan --tie fifo
```

只输出完整文本结果、不加载绘图库、不生成图片（仍输出坐标路径、步数、总代价、扩展数和耗时）：

```
python astar.py --no-plot
```

自定义地图可以直接修改 `demo_map` 中的数组和坐标；也可以自行创建 JSON 文件，例如：

```
{"grid": [[0, 0, 0], [0, 1, 0], [0, 0, 0]], "start": [0, 0], "goal": [2, 2]}
```

```
python astar.py --input map.json --movement 8 --no-plot
```

JSON 文件是可选输入，不属于默认三个提交文件。默认不传参数即可复现提交的默认运行图。左侧显示用户选定方案；右侧显示另一邻域，并采用该邻域的默认启发式。两幅路径图使用相同的同优先级策略：`--tie fifo` 时均为 fifo，默认均为 low_h。自定义启发式时，两图的启发式仍可能不同，因此不能把扩展数差异完全归因于邻域。

图片底部的策略对比表分别列出四邻域、八邻域的四种合法启发式及各自 low_h、fifo 的扩展数，共展示 16 组配置的实际计算结果。该表不会完整打印到终端；使用 `--no-plot` 或缺少绘图库而跳过绘图时，也不会执行这部分额外对比。

7 实际运行结果与分析

提交图使用默认 15×20 地图，起点 $(1,1)$，目标 $(13,18)$。三道竖向墙位于第 5、10、15 列，分别包含缺口并保留从墙端绕行的路线。图中的蓝色区域表示已扩展格，深色为障碍物，橙色为最终路径。

邻域	启发式	最短代价	扩展数：low_h	扩展数：fifo
四	零启发式	29.000	249	249
四	曼哈顿	29.000	30	120
四	欧氏	29.000	143	161
四	切比雪夫	29.000	163	190
八	零启发式	23.728	249	249
八	Octile	23.728	63	72
八	欧氏	23.728	83	87
八	切比雪夫	23.728	118	122

扩展数按有效出堆次数计算，包含起点和目标；不包含被跳过的过期堆条目。表格按三位小数显示代价，算法内部保留浮点精度。

默认四邻域路径有 30 个坐标点、29 步，代价为 29。图中八邻域路径为 20 步，其中正交 11 步、对角 9 步，因此代价为 $11 + 9\sqrt{2} \approx 23.727922$。

在同一邻域下，所有策略得到相同最短代价。该示例中，曼哈顿和 Octile 对应的搜索范围小于零启发式；同样的四邻域曼哈顿配置，小 h 优先扩展 30 次，而按入队顺序扩展 120 次，说明同 f 节点的选择会影响搜索量。

八邻域允许更多动作，所以最优代价低于四邻域。这反映移动模型的差异，不能据此宣称八邻域算法本身更高效。以上节点数是该固定案例的结果，不是对任意地图的结论。耗时会随机器与系统负载变化，终端给出的单次耗时仅作运行参考。

8 正确性验证

交付前使用独立 Dijkstra 参考实现核对算法。参考实现自行生成合法邻居，不调用本代码的 `neighbors` 或 `estimate`，减少共享错误掩盖问题的可能性。

采用随机种子 `20261013` 生成 100 张 6×7 地图，障碍物生成概率为 0.28，并确保起终点为空地。每张地图验证四邻域的四种启发式、八邻域的四种启发式及各自两种同优先级策略，共 1600 组搜索。逐一核对可达性、最短代价、路径首尾、每步合法性、障碍物避让和累计代价，全部通过。

另外验证空地图、非矩形地图、非 0/1 单元格、非法坐标、越界、障碍物起终点、不合法策略组合、起终点相同、对角防穿角、输入地图保持不变及重复搜索路径一致。独立验证脚本为开发时使用的临时工具，未额外作为第四个交付文件。

本次代码修订后共通过 12 项测试，包含上述 1600 组独立最短路径核对。命令行检查覆盖 Octile 参数提示、非法 JSON 与地图输入、输出目录不存在、输出路径为目录、--no-plot 忽略图片路径、BOM 编码 JSON 读取，以及缺少绘图库时保留文本结果；绘图检查覆盖四 / 八邻域和两种同优先级策略，核对两幅图的标题、实际指标与策略一致性。

9 复杂度，优点与局限

设地图格子总数为 N，可达格子数为 V，合法边数为 E。输入校验需 O(N)。栅格中每个节点最多 8 条出边，因此 E = O(V)。在本程序的一致启发式配置及理想精确代价模型下，节点有效扩展至多一次，堆操作对应边松弛与过期条目处理，搜索时间为 O((V+E) log V)，即 O(V log V)，额外搜索空间为 O(V+E)，即 O(V)。路径回溯需 O(L)，L 为返回路径的坐标点数。

实际实现使用浮点代价，微小舍入差异可能引入额外更新；程序允许重新入堆，因此不能将上述有效扩展次数结论泛化到任意非一致启发式或任意浮点输入。一般允许重复改进的情形，若入堆总数为 P，堆处理可按 O(P log P) 时间及 O(P) 堆空间描述。绘图另需 O(N) 的地图副本，不计入纯搜索空间。

优点：实现直观、单文件容易运行和审阅；启发式与邻域选择明确；具有输入校验和对角碰撞限制；输出坐标序列，可用真实结果比较策略。

局限：只考虑静态障碍物和点状移动对象；不建模车辆尺寸、转弯半径、速度、动态障碍物或道路规则；结果依赖栅格分辨率，大地图的搜索和绘图可能占用较多内存。用于真实车辆时，还需障碍物膨胀、运动学约束和动态重规划，本作业不把这些额外功能混入基础 A* 实现。